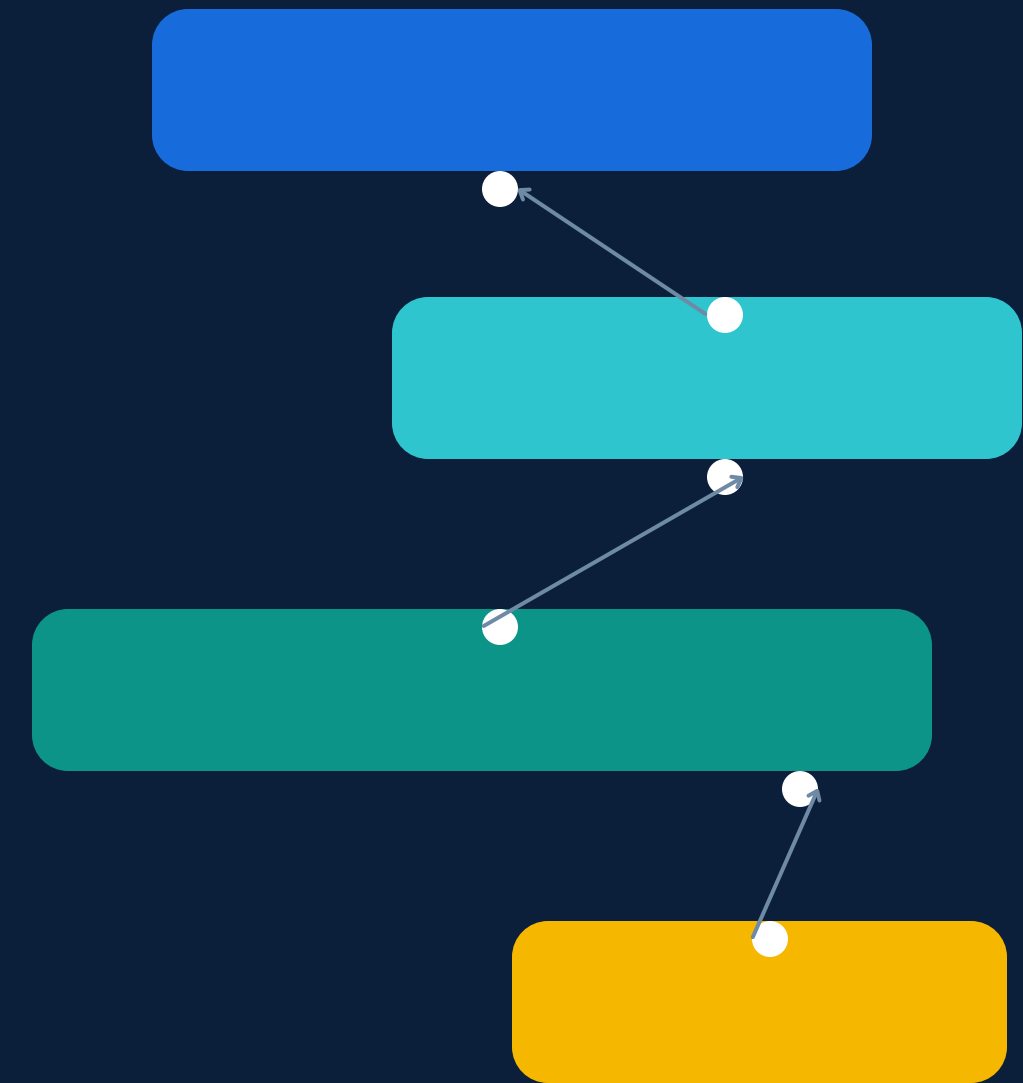


TraceLens

Explainable code-change impact intelligence

A developer tool that shows what changed, what may be affected, and what to test before merging.



Theme: Developer Tooling

Understanding Code Changes and Debugging

The problem: a small diff can hide a large risk

Developers see changed lines, but often lack context about the codebase-wide effect.

A developer changes a token parser

The code edit looks small. The parser may still affect login, session validation, API clients, and tests.

Reviewers must trace dependencies manually

They search files, follow imports, guess test coverage, and compare the patch with the intended requirement.

Teams merge with blind spots

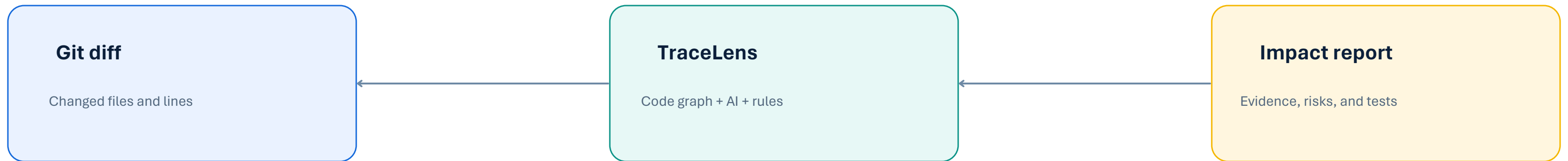
Missing edge-case tests or an unintended interface change can create regressions after the pull request is approved.

Who feels this most?

Student teams learning collaboration • New open-source contributors • Developers entering an unfamiliar codebase

TraceLens: an evidence-linked impact report

TraceLens turns a pull request into a clear explanation that a developer can verify.



The report answers the questions a reviewer asks first

What changed?

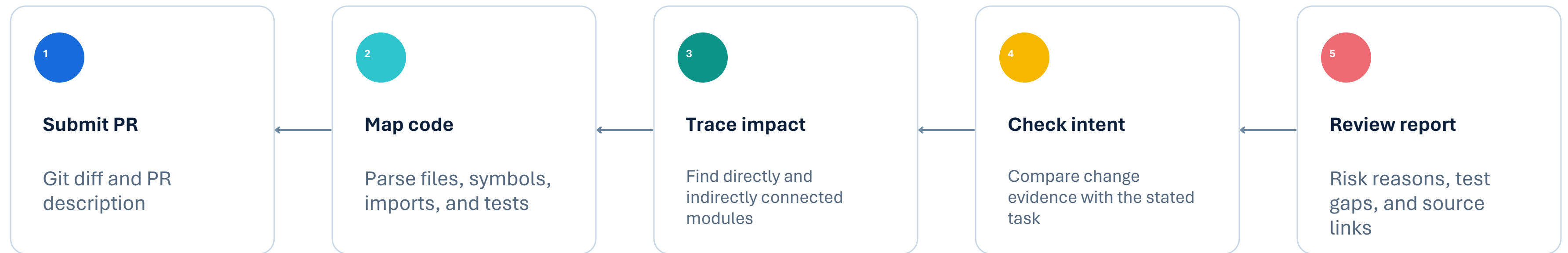
Which components may be affected?

What should we test?

Every finding points to a code location, dependency path, or test file so the team can inspect the evidence.

End-to-end workflow

TraceLens runs at pull-request time and gives the developer a focused review checklist.

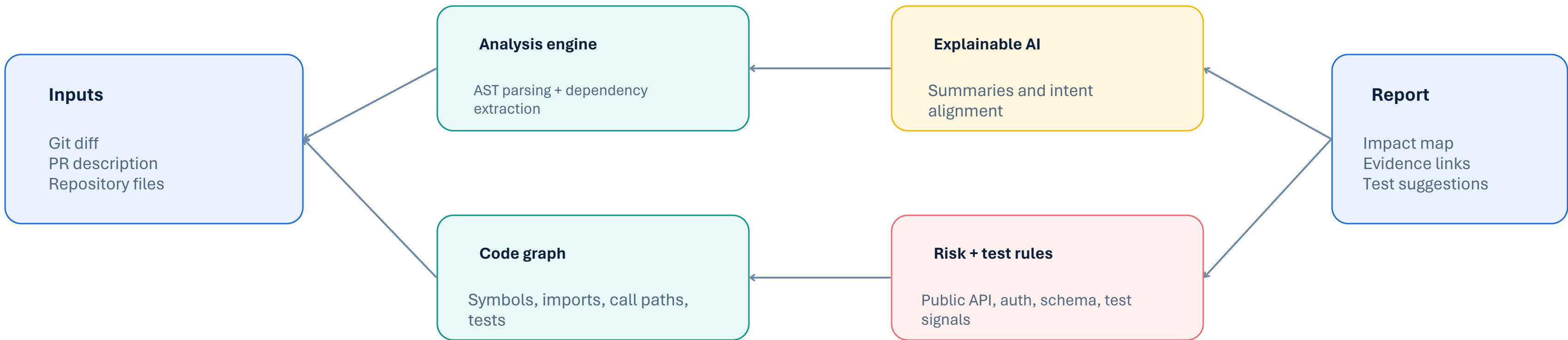


Outcome: the developer knows where to review and what to validate before merge.

The workflow narrows attention to relevant code paths instead of asking people to inspect the entire repository.

System architecture

The design combines deterministic code analysis with an AI layer that explains only the available evidence.



Reliability principle

TraceLens separates confirmed graph facts from AI inferences and attaches a confidence level and supporting evidence to each conclusion.

Proposed technology stack

A practical MVP can focus on JavaScript and TypeScript repositories before expanding to other languages.

Layer	Technology	Role in TraceLens
 Developer interface	React / Next.js	Pull-request report and interactive impact map
 Backend API	Python FastAPI or Node.js	Run analysis and generate reports
 Code intelligence	Git + Tree-sitter + AST parsers	Read diffs and extract symbols, imports, calls, and tests
 Dependency analysis	NetworkX or Neo4j	Store and traverse the code relationship graph
 AI layer	LLM with retrieval over code evidence	Explain change intent without losing traceability
 Integration	GitHub REST / GraphQL API	Fetch PR metadata, commits, and repository context
MVP scope	Analyze a pull request, build an import/function graph, locate related tests, and produce an evidence-linked report.	

What a TraceLens report looks like

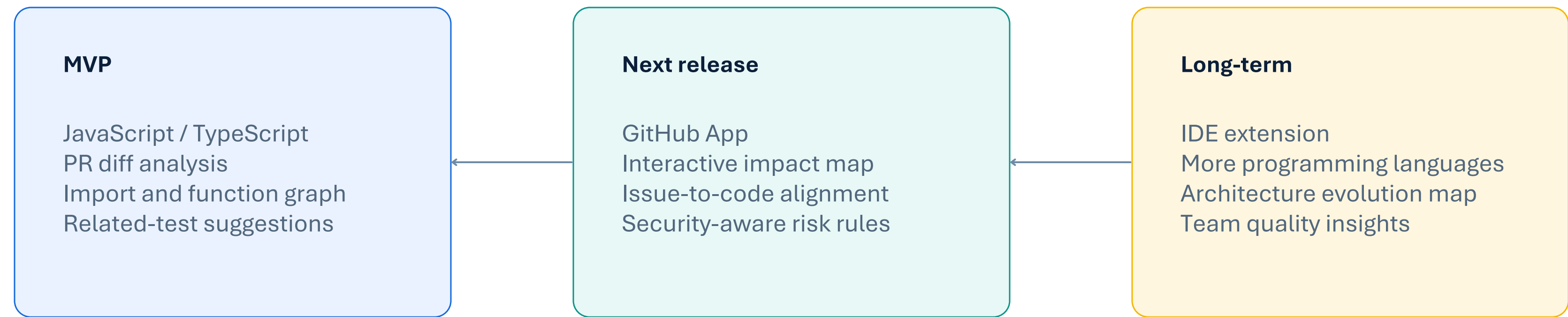
The report prioritizes a reviewer's attention and makes each warning easy to inspect.

Pull request: Update authentication token format		HIGH IMPACT
Change summary	Evidence	
Token parser now accepts a revised expiry field.	3 code paths 2 existing tests 1 missing edge case	
Affected paths		
auth/tokenParser.ts → loginController.ts → sessionMiddleware.ts		
Test gap	Why high risk?	
Add an expired-token test for the revised token format.	Authentication is shared across multiple request paths.	

Each statement links back to code, dependencies, or tests. The developer decides what to do with the evidence.

MVP and future scope

Start with a narrow, demonstrable workflow and expand once the core impact report works reliably.



Expected value

TraceLens helps student teams and developers understand changes faster, target tests better, and review unfamiliar code with evidence instead of guesswork.